Using a Pluggable Authentication Module for Verifying User Identities

Pluggable authentication modules help to separate the tasks of authentication from applications. Learn how to build a login mechanism that is API based and uses a custom PAM module for authenticating user identities.

Pluggable authentication modules (PAM) help with user identification in Ubuntu or any other Linux flavour (through the user interface, terminal or other means) by separating the general and specific tasks of authentication from applications.

Many programs—like *login*, *gdm*, *sshd*, and *ftpd*—need to verify user identities, but there are multiple ways to accomplish this. For instance, a user may provide a user name and password, which can be stored locally or managed through LDAP (Lightweight Directory Access Protocol) or Kerberos. Alternatively, a user can authenticate with a fingerprint or certificate. Requiring each application developer to implement new authentication methods is inefficient. Instead, PAM allows applications to rely on its libraries, leaving the details to authentication experts. PAM's pluggable design lets different applications apply different authentication tests, while its modularity makes it easy to add new methods through additional libraries.

Let's now create a login mechanism that is API based. The PAM module will be sending the request to the API server for validation and will authenticate based on the output received from the API. We can create a new login scheme with this sample PAM module and server.

Architecture

For getting the customisable login, we will be adding a new PAM module to the host PC to read the password from the user and send it to the cloud authentication server (hosted in LAN/WAN). If the server says the password is valid in the cloud, the latter will reply with the user name, so that the host PC can check if the user name received matches with the

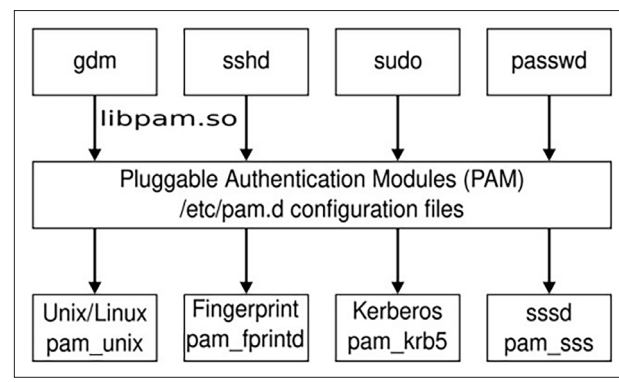

Figure 1: PAM's pluggable design

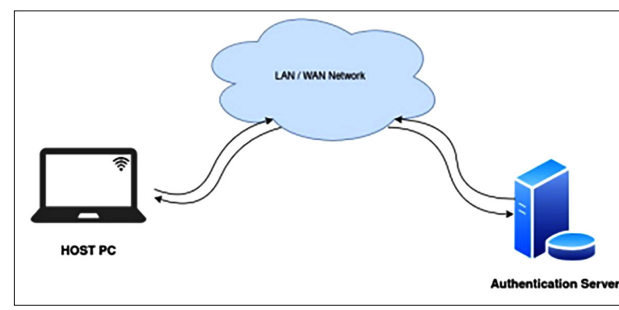

Figure 2: Cloud authentication